Efficient Intrusion Detection in AMI Systems Based on Federated Semi-Supervised Learning

Zhuoqun Xia, Haidong Tang, Zhenzhen Hu, *Member, IEEE*, and Hongmei Zhou, *Member, IEEE*

***Abstract*—The advanced metering infrastructure (AMI) network is the most critical part of a smart grid, and it faces serious security challenges from network attacks. Federated learning (FL) is a common method for addressing network security problems; however, it has several shortcomings, such as difficulty in obtaining labeled data and high communication costs. Therefore, in this paper, an efficient intrusion detection method based on federated semi-supervised learning is proposed to improve the efficiency of AMI network intrusion detection and reduce communication overhead. First, an intrusion detection framework based on federated distillation (FD) is established to address intense assumption dependency problems and reduce communication overhead. Then, an efficient intrusion detection algorithm is used to improve the classification performance. Finally, the designed deep convolutional generative adversarial network (DCGAN) model is used to obtain high-quality sample data. The experimental results show that the scheme achieves an accuracy of 99.58%, a communication overhead of 75 MB, and a reduction in the false-positive rate of 6%.**

***Index Terms*—Advanced metering infrastructure (AMI), intrusion detection, federated distillation (FD), semi-supervised learning, deep convolutional generative adversarial network (DCGAN).**

I. Introduction

IN RECENT years, the vigorous evolution of smart grids (SGs) has yielded numerous application services, such as demand response (DR), advanced metering infrastructure (AMI), and real-time price (RTP), which has been very beneficial for customers and grid companies [1]. An AMI is a typical application service for smart grids. The AMI consists of group smart meters, concentrators, and data centers. In addition, AMI enables bidirectional communication between customers and utilities [2]. However, massive data transmission imposes a considerable burden on the communication of AMI networks. Fortunately, fifth-generation (5G) communication technology can meet wide coverage, large connectivity, and low latency requirements. Therefore, using 5G communication technology in the AMI network is an efficient solution [3]. Smart grids (SGs) and wireless technology enable AMI to integrate more flexibly with smart meters and more services/applications. However, AMI systems are widely distributed and vulnerable to various security attacks, such as denial of service (DoS), false data injection (FDI), and user-to-root (U2R) attacks [4]. In 2015, a distributed denial of service (DDoS) attack on Ukrainian AMI systems affected 225000 consumers [5]. In 2020, the Energias de Portugal (EDP) electricity company suffered from Ragnar Locker Ransomware attacks, which resulted in economic losses of $10.9 million [6]. Therefore, network security attacks have a history of leading to severe financial losses and hindering people's daily lives.

Intrusion detection systems (IDSs) can intercept network traffic and trigger real-time alerts to improve the performance of AMI networks in monitoring and detecting failures. Recent studies have focused on deep learning (DL)-based intrusion detection systems to improve the accuracy of AMI intrusion detection. Some authors have proposed AMI network intrusion detection schemes that combine feature downscaling and modified long short-term memory (LSTM) networks to increase classification accuracy. However, this approach has flaws, such as low data privacy, low efficiency, and high latency [7], [8], due to the possibility of exploiting privacy-sensitive network traffic data. To improve communication security, several authors have investigated federated learning (FL)-based intrusion detection methods for AMI networks [9], [10], [14]. However, these methods have two drawbacks. First, large numbers of AMI devices upload model parameters to cloud servers, resulting in heavy communication burdens. Second, existing FL-based intrusion detection methods rely on strong assumptions (adequate available traffic samples, labeled samples, and having all samples available for model training). In these methods, the client uploads the overall model to the cloud server rather than using the Federated Distillation (FD) model parameters. Therefore, the FDs are widely used to reduce communication costs [11], [12], [13]. In the real world, these unlabeled data are rarely used in supervised learning procedures due to the high cost of data labeling, lack of most of the data being labeled, limited stored data, missing and unbalanced data, etc. [15]. Semi-supervised learning has gained widespread attention in recent years because it can efficiently address insufficiently labeled data.

Although the above literature has addressed the relevant issues to some extent, there are still two significant shortcomings: (1) Several studies have been performed on intrusion detection FL-based methods in AMI networks, however, not many

Received 15 February 2024; revised 9 April 2025; accepted 14 April 2025. Date of publication 21 April 2025; date of current version 25 August 2025. This work was supported in part by the National Natural Science Foundation of China under Grant 52177067 and Grant 62402209 and in part by the Hunan Natural Science Foundation under Grant 2023JJ30052. Recommended for acceptance by Prof. Xiaowen Chu. *(Corresponding author: Zhenzhen Hu.)*

Zhuoqun Xia, Haidong Tang, and Hongmei Zhou are with the College of Computer, Changsha University of Science and Technology, Changsha 410114, China (e-mail: xiazhuoqun@csust.edu.cn; 1031564607@qq.com; zhouhongmei@stu.csust.edu.cn).

Zhenzhen Hu is with the College of Computer Science, University of South China, Hengyang 421001, China (e-mail: hzz88@hnu.edu.cn).

Digital Object Identifier 10.1109/TNSE.2025.3562751

of the studies have considered the attack detection accuracies and communication overheads, which significantly reduces the practical application potential of the proposed model. (2) Several existing works on intrusion detection modeling have focused on using or modifying the structure of a neural network to achieve a high detection rate but have ignored the complexities of data collection and use. However, the processing of datasets may sometimes be more time-consuming than the study of detection models. Therefore, datasets must be automatically processed for AMI-IDS research.

This paper proposes an effective intrusion detection method for AMI networks based on federated semi-supervised learning, known as FSSL-IDS, to address these challenges. The main research contributions of this paper are as follows:

- We propose a distributed intrusion detection framework based on FD to provide efficient communication methods for AMI networks. The framework allows AMI devices to participate in creating global detection models by exchanging model outputs. The switched model output reduces the communication overhead for intrusion detection in AMI networks and enables high detection performance.
- We create an intrusion detection model based on the DCGAN. The limited labeled data from the cloud server can be supplemented by the synthetic data produced by the DCGAN, which has the advantages of efficient feature extraction capabilities, fast training speeds, and high detection accuracies in implementing intrusion detection in the AMI network. Moreover, the DCGAN addresses the issues of missing categories, imbalances, and limited labeled data on the cloud server side.
- The PyTorch framework is used to conduct simulation experiments on the proposed model. These experiments verify that our proposed FSSL-IDS model not only has high accuracy but also reduces the FL communication overhead. The introduction of the DCGAN model fully utilizes the AMI dataset, which endows the model with high practical value.

The remainder of this paper is structured as follows. In Section II, related work is discussed. In Section III, the preparatory knowledge is presented. In Section IV, the FSSL-IDS detection framework, the DCGAN intrusion detection model, and the federated semi-supervised learning intrusion detection technique are introduced. In Section V, the performance of the FSSL-IDS is experimentally evaluated. The conclusions of this paper are presented in Section VI.

II. Related Work

A. DL-Based IDSs for AMI Networks

In response to insufficient machine learning (ML), some researchers have investigated deep learning-based intrusion detection for AMI networks. Several works have demonstrated that DL outperforms ML in terms of AMI network intrusion detection. For example, Zheng et al. [17] proposed a depth-and-breadth convolutional neural network (CNN) model to identify power theft in a smart grid. To detect intrusions into smart grids, Taghavinejad et al. [18] proposed a hybrid deep neural network to improve detection performance by incorporating multiple calssification and regression tree (CART) decision tree classifiers. Vijayanand et al. [19] used a multilayer deep learning algorithm to analyze smart meter traffic for accurate attack detection. He et al. [20] proposed an AMI network intrusion detection method using the conditional deep belief network (CDBN), which can identify a false data injection via several external conditions. Xiao et al. [21] combined self-encoders with convolutional neural networks and reduced the dimensionality of the data to reduce redundant feature interference. Xu et al. [22] analyzed abnormal data in AMI networks via a bidirectional gated recurrent unit (BiGRU). Compared with LSTM, the BiGRU is more effective and accurate and has a lower false alarm rate. Alshede et al. [36] proposed an integrated voting algorithm based on random forest and convolutional neural networks to classify multiclass attack categories. Alsirhani et al. [37] proposed a smart grid intrusion detection strategy that combines deep learning and signature techniques. After the dataset is preprocessed, the feature values are extracted, and the African culture optimization algorithm is used to organize the feature selection. Then, deep belief network (DBN)-LSTM is used to identify attacks. Alabassi et al. [38] proposed a deep learning-based system for critical infrastructure cyber-attack detection and location recognition by building new representations and modeling the system's behavior via multiple layers of autoencoders.

In AMI intrusion detection, deep learning improves accuracy and multiclassification while reducing overfitting. However, there is a reliance on centralized data, which poses privacy issues and network strain.

B. FL-Based IDS for AMI Networks

Federated learning is a distributed learning system in which clients work together to train models under the supervision of a cloud server. Training data can be stored locally on the client, and the cloud server must be informed only of local distributed model training parameters [23], [24]. This method lowers the costs and privacy hazards associated with centralized ML methods. Federated learning is a viable method for detecting AMI intrusions because it can help address data privacy concerns.

Recently, Sun et al. [8] suggested an intrusion detection solution for AMI networks that employs Transformer models for hierarchical federated learning training. The system achieves collaborative training of shared intrusion detection models for smart meters while protecting privacy and reducing communication costs. In addition, Mirzaee et al. [9] presented a federated intrusion detection solution for AMI networks based on deep neural networks (DNNs), which achieves secure and private AMI systems by monitoring AMI network traffic with federated deep neural network detection techniques and constructing a distributed IDS architecture. Xia et al. [10] suggested a client-based paradigm for effective AMI network intrusion detection. The framework comprehensively considers the client's security performance and computing power to achieve efficient intrusion detection. It uses a DBN based on an attention mechanism, thereby reducing the communication overhead. Liang et al. [25] proposed a federated learning approach for AMI intrusion detection, with local DNN model

TABLE I
NOMENCLATURE

Symbol	Meaning
θ_0,θ_t	Initial model parameters and locally updated model parameters from the t-th round
D^k,$D^{m,c}$,D^p	Dataset of client k, unlabeled dataset of concentrator m, labeled dataset of cloud server
x_i^k,$x_i^{m,c}$,x_j^p	Sample feature vectors from D^k,$D^{m,c}$,D^p respectively
y_i^k,y_j^p	One-hot encoded sample labels from D^k,D^p
N^l,$N^{k,l}$	Number of samples belonging to label l in client k, and Total number of categories
K	Total number of clients
Z_i	Noise generated by Gaussian distribution
X_G	Fake data generated by the generator
D'	Input data to the discriminator
$\hat{p}_{i,m}$,$\hat{p}_{i,s}$	Local logits and global logits
$\psi\left(D^{m,c}\middle\vert\theta_0\right)$	Cross entropy loss function
η,α	Model update and discriminator learning rate.
k_m	Number of fake data generated by the generator
Y_z,Y_R	Labels for fake data generated by the generator and labels for real data
φ,φ_1,ν	Accuracy, optimal accuracy threshold, and similarity threshold

training on concentrators and a data center aggregating and distributing a global model. Wen et al. [26] developed a federated learning framework for collaborative privacy-preserving power theft detection in smart grids by employing a secure communication protocol and temporal convolutional network (TCN) model. Wang et al. [39] proposed an intrusion detection method based on an AMI network federated learning client security. In addition, adaptive weights have been used to resist poisoning attacks. Naeem et al. [40] proposed a novel FL-empowered semi-supervised active learning (FL-SSAL) security orchestration framework for a label-at-client scenario that used labeled and unlabeled samples to detect attacks in the network infrastructure.

However, supervised approaches in AMI intrusion detection cannot utilize large amounts of unlabeled data. This paper proposes a method using a DCGAN within FL to improve detection performance by leveraging both labeled and unlabeled data, and achieves high-precision classification. Additionally, federated distillation is employed to optimize the communication overhead, resulting in a highly accurate and efficient model.

III. Preliminary Knowledge

A. Federated Distillation

The parameters andexplanations used in this paper are as shown in Table I. Federated learning was proposed by Google in 2016 [7], and its emergence addressed the capability of attackers to steal or tamper with raw data during the upload process. The mainstream federated averaging (FedAVG) algorithm ensures that the client shares only the model parameters but not the local raw data, thus ensuring the privacy of the client's local data. Moreover, FD can shrink the model. The client participating in FD does not need to upload model parameters; only the average predicted values of the labels, namely, the local model output (local logits), need to be uploaded. After the Softmax function operation, the local model output is a normalized vector value, which is much smaller than the size of the local model parameters, largely reducing the communication overhead of the entire training process [27], [28]. Suppose that the local dataset of client k is $D^k = (x_i^k, y_i^k)|i = 1, 2, \ldots, n$, where n denotes the number of samples in dataset D^k. Then, D^k can be divided into l subsets according to the labels, that is, $D^k(D^k = D^{k,0} \cup D^{k,l} \cup \ldots \cup D^{k,L-1})$. For any subset $D^{k,l}$, $D^{k,l} = \{(x_i^{k,l}, y_i^{k,l}), i = 1, 2, 3, \ldots, n\}$. The label of each sample is a one-hot vector; for example, the one-hot vector of label L can be expressed as $y^{ik}, l = \{y_{i,0}^{k,l}, y_{i,1}^{k,l}, \ldots, y^i, l^k, l, y_{i,L-1}^{k,l}\}$, where $y_{i,l}^{k,l} = 1$ and the other values are equal to 0. The FD algorithm consists of the following six main steps [11], [12], [13]:

1) *Training phase:* The client k performs the same procedures as in federated learning local training to train its local model ω^k with the dataset D^k.
2) *Prediction Phase:* The local average logit vector, which is calculated for client k and displayed below, represents the average value of each label:

$$\hat{y}_i^{k,l} = F\left(x_i^{k,l}|\omega^k\right) \tag{1}$$

$$\bar{y}^{k,l} = \frac{1}{N^{k,l}} \sum_{i=1}^{N^{k,l}} \hat{y}_i^{k,l} \tag{2}$$

If client k has no samples with label l, namely, $D^{k,l} = \emptyset$, then the label l's local average logit vector is 0, that is, $\bar{y}^{k,l} = 0$.
3) *Upload phase:* Each client uploads $\{\bar{y}^{k,0}, \bar{y}^{k,l}, \ldots, \bar{y}^{k,L-1}\}$ to the cloud server.
4) *Aggregation phase:* The client receives the global average logit vector via broadcast from the cloud server after performing global aggregation.

$$\bar{y}^{s,l} = \frac{1}{K} \sum_{k=1}^{K} \bar{y}^{k,l} \tag{3}$$

5) *Update Phase:* Each client calculates a new local average logit vector based on the global average logit as follows:

$$\bar{y}^{k,l} = \frac{1}{N^l - 1}\left(N^l \times \bar{y}^{s,l} - \bar{y}^{k,l}\right) \tag{4}$$

where each sample x_i with true label l has a local average logit vector $\bar{y}_i^{k,l}$ equal to $\bar{y}^{k,l}$.
6) *Iteration phase:* Client k trains its model using the true labels and the new local average logit vector.

B. Generating an Adversarial Network

The deep learning network called the generative adversarial network (GAN) uses an inverse model [29]. Fig. 1 depicts the network structure. Generators train themselves to generate artificial data, whereas discriminators are trained to distinguish between raw and generated data. The performances of the two networks can be incrementally improved as they operate in parallel. After n iterations, the discriminator learns to identify the data source, whereas the generator learns to produce data similar to the original data. The GAN function is as follows:

$$\begin{aligned} \min_G \max_D = {} & E_{x\,P_{data}}\left(log D_i(x)\right) \\ & + E_{z\,P_z}\left(log\left(1 - D_i\left(G_i(z)\right)\right)\right) \end{aligned} \tag{5}$$

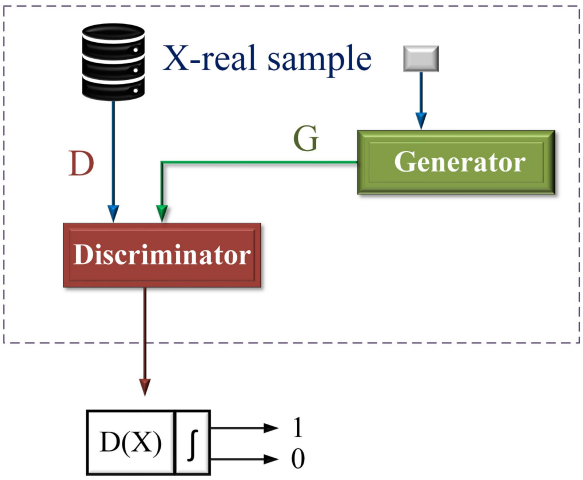

Fig. 1. Generating an adversarial network.

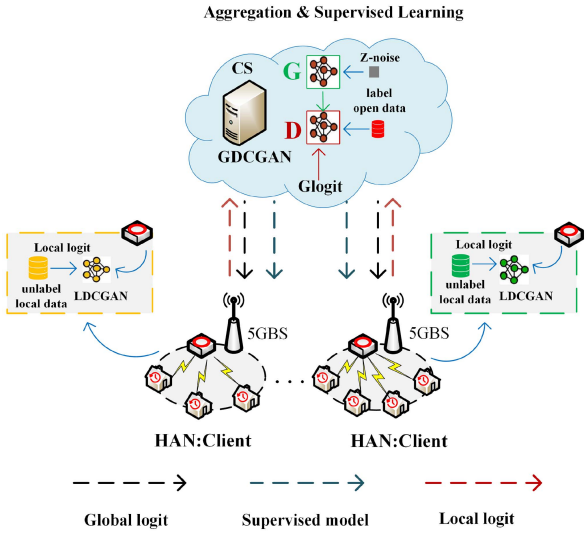

Fig. 2. FSSL-IDS detection framework.

where $G_i(z)$ represents the synthetic data generated by the generator, and the discriminator output is $D_i[0,1]$, which expresses the likelihood that the data are authentic or a copy. The generator aims to reduce the possibility that the discriminator classifies the data source as $1 - D_i(G_i(z))$. The discriminator's goal is to maximize the probability of data source recognition, that is, $1 - D_i(G_i(z))$ and $D_i(x)$. The generator's input is the noise signal, which is a random vector of size k that follows the normal distribution and provides data that are similar to the training data. The generator and the training samples serve as the discriminator's input so that the discriminator can distinguish between them.

IV. The Proposed FSSL-IDS Scheme

Joint training by exchanging equipment local FL model parameters is a common practice in the AMI intrusion detection field and provides benefits for data privacy protection [29], [30], [31], [32], [33]. However, in traditional FL-based intrusion detection methods for AMI networks, AMI devices require communication overhead proportional to the model size when training is performed. In addition, in actual AMI networks, the data obtained from the AMI device are usually partially labeled or may be completely unlabeled. To address the problems mentioned above, we propose a method based on the federal AMI network intrusion detection method.

A. FSSL-IDS Detection Framework

In this section, our detection framework and main ideas are described, as shown in Fig. 2. The AMI detection framework proposed in this paper consists mainly of a data center (i.e., cloud server), a concentrator, and a 5G base station, which are described in detail below.

1) Data Center: The data center is a cloud server deployed by an electric utility that is in charge of gathering, analyzing, and processing the data, serving as the federated learning counterpart of the central server. Before training begins, considering that the standard samples are the majority, and that attack samples are the minority, the generator using the DCGAN model generates synthetic data based on the original labeled data, feeds it to the discriminator for supervised learning along with the original labeled data, and finally broadcasts the initialized DCGAN model to the centralizer. In addition, after each round of communication, the DCGAN supervised model is adjusted using the global model output, which is created by incorporating the local model output from the cloud server. Finally, the cloud server broadcasts the global model output and the DCGAN supervisory model back to the concentrator.

2) Concentrator: The concentrator is the local client for FL training. This device collects smart meter power data from residential customers and requires privacy and security protection. Smart meters communicate with home appliances through the home LAN to collect data. The power data are regularly delivered to a nearby concentrator for storage and co-training. In each round of training, the concentrator uses local unlabeled data for local updates, obtains local model outputs, and performs category prediction on the unlabeled data. The output of the local model is then transferred to the cloud server.

3) 5G Base Stations: 5G base stations are deployed near the concentrator to improve the spectral efficiency of communication between the concentrator and the cloud server via 5G network slicing technology and to provide low-latency end–to-end connectivity. The concentrator generates the local model output through federated distillation learning and transmits it to the cloud server for aggregation over the 5G core network.

B. Federated Semi-Supervised Intrusion Detection Algorithm

In this section, we detail the design of the federated semi-supervised learning algorithm, as shown in Fig. 3. Specifically, a small publicly available labeled dataset is first introduced, and the cloud server uses this labeled dataset for initial training. Before training, a generator network using the DCGAN model generates synthetic data based on the original labeled data; it

Algorithm 1: FSSL-IDS Algorithm.

Cloud Server:
Initialize the supervised classification model θ_0 // Global model parameters;
Distribute the supervised model to randomly selected clients;
Update:
for *each client* m ***in parallel*** **do**
  Update the local model parameter θ_t;
  $\theta_t \leftarrow \theta_0 - \eta \nabla \Psi \left(D^{m,c} | \theta_0\right)$;
Prediction:
for *each client* m ***in parallel*** **do**
  Calculate local logits $\hat{p}_{i,m}$;
  $\hat{p}_{i,m} = F\left(x_i^{m,c} | \theta_t\right)$;
Upload:
Each client uploads the local logits $\hat{p}_{i,m}$;
Aggregation:
The cloud server aggregates the logits to create the global logit $\hat{p}_{i,s}$;
$\hat{p}_{i,s} = \frac{1}{M} \sum_{m=1}^{M} \hat{p}_{i,m}$;
$\hat{p}_{i,s} \leftarrow S\left(\hat{p}_{i,s} | T\right)$;
Supervised Learning:
The cloud server supervises learning the global logit $\hat{p}_{i,s}$;
Update the supervised classification model θ_{t+1};// Global Model Parameters
$\theta_{t+1} \leftarrow \theta_{t+1} - \eta \nabla \psi \left(D^p, \hat{p}_{i,s} | \theta_{t+1}\right)$;
Broadcast:
Broadcast $\hat{p}_{i,s}$ to randomly selected clients;
Distillation:
for *each client* m ***in parallel*** **do**
  Update the local model parameter θ_t;
  $\theta_t \leftarrow \theta_t - \eta_{dist} \nabla \psi \left(D^{m,c}, \hat{p}_{i,s} | \theta_t\right)$;
Steps 4–27 are iterated for multiple rounds until convergence.

then feeds these synthetic data together with the original labeled data into the discriminator for supervised learning. The local concentrator is trained locally via a large amount of unlabeled data via the FD algorithm after receiving the supervised model from the cloud server to determine the class to which each sample in the unlabeled data belongs, thereby assisting the cloud server in performing iterative training for supervised learning.

We assume that there are M concentrators, and that each concentrator $m \epsilon \{1, 2, 3, \ldots, M\}$ has a local unlabeled dataset $D^{m,c} = \{x_i^{m,c} | i = 1, 2, 3, \ldots, n\}$. The cloud server has a small number of publicly available labeled datasets $D^p = \{(x_j^p, y_j^p) | j = 1, 2, 3, \ldots, N^p\}$. Intrusion detection is similar to a classification task. In this section, there are assumed to be L categories, and y_j^p is assumed to be a one-hot vector. In addition, each concentrator receives a supervised classification model θ_0 broadcast by the cloud server before local unsupervised training.

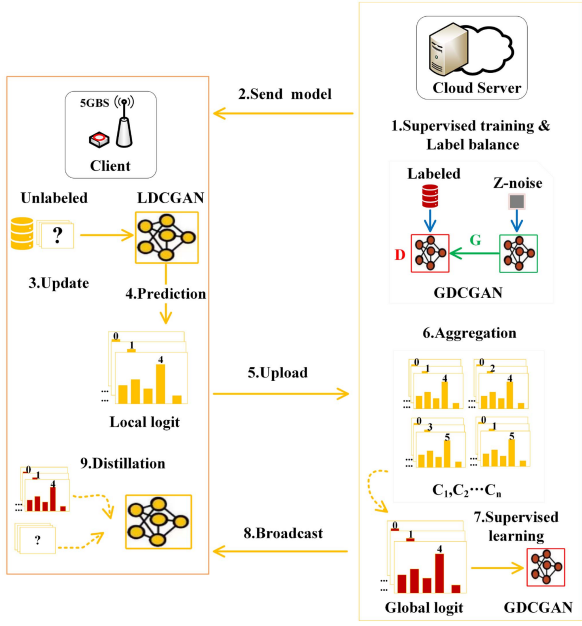

Fig. 3. Communication process for federated semi-supervised learning.

This model is trained on a small publicly available labeled dataset. The proposed algorithm is described in detail below.

1) Supervised Training and Label Balancing Phase: First, the cloud server trains the supervised classification model θ_0 with a small number of publicly available labeled datasets. Considering a class imbalance problem in labeled datasets, synthetic data are produced from the original labeled data before training via the generator network of the DCGAN model. The discriminator for supervised learning is then given both synthetic data and original labeled data. The detailed formula is as follows:

$$Z_i = GaussianNoise(b) \tag{6}$$

$$X_G = G\left(\omega, z\right) | z \epsilon Z_i \tag{7}$$

$$D' = X_G + D^p \tag{8}$$

$$\theta_0 = Adam\left(\triangle_\theta \frac{1}{m} \sum_{i=1}^{m} \left(-D\left(G_\theta(z)\right), \theta, \alpha\right)\right) \tag{9}$$

where b denotes a random number, where $Adam$ denotes the optimization function, where D denotes the discriminator, and where G denotes the generator.

2) Distribute Model Parameter Phase: The cloud server distributes the supervised classification model θ_0 to the local concentrator through the 5G core network.

3) Local Update Phase: Based on the stochastic gradient descent algorithm, each concentrator is trained unsupervised with a substantial amount of local unlabeled data, and the local update model θ_t is calculated as shown below:

$$\theta_t = \theta_0 - \eta \nabla \psi \left(D^{m,c} | \theta_0\right) \tag{10}$$

In the intrusion detection problem, the cross-entropy loss function is generally used, and $\psi(D^{m,c}|\theta_0)$ is calculated as follows:

$$\psi\left(D^{m,c}|\theta_0\right) = -\sum_{i=1}^{n} logF\left(x_i^{m,c}|\theta_0\right) \tag{11}$$

4) Prediction Phase: Concentrator m uses federal distillation to calculate the local model output $(local\ logits)$, which represents the inferred probability of each data sample being classified into each category. To achieve category prediction of unlabeled data, a local unsupervised model based on what was learned in the previous stage must be used. The $local\ logits$ values predicted by each concentrator m are calculated as follows:

$$\hat{p}_{i,m} = F\left(x_i^{m,c}|\theta_t\right) \tag{12}$$

where the matrix $\hat{P}_m$ is used to denote the set $\hat{p}_{i,m}|i = 1, 2, 3, \ldots, n$.

5) Uploading Phase: The concentrator uploads the local model output $\hat{P}_m$ to the cloud server, unlike in traditional FL, which uploads the model parameters θ_t.

6) Aggregation Phase: The cloud server performs entropy reduction, which primarily accelerates and stabilizes the FD process, to combine the uploaded local model output with the global model output $\hat{p}_{i,s}$, which is calculated as follows:

$$\hat{p}_{i,s} = \frac{1}{M}\sum_{m=1}^{M}\hat{p}_{i,m} \tag{13}$$

$$\hat{p}_{i,s} = S\left(\hat{p}_{i,s}|T\right) \tag{14}$$

where S is the Softmax function at temperature T.

7) Supervised Learning Phase: The cloud server takes the global model output $\hat{p}_{i,s}$ as the input of the DCGAN model and helps the DCGAN model be supervised and trained again.

$$\theta_{t+1} = \theta_{t+1} - \eta\nabla\psi\left(D^p, \hat{p}_{i,s}|\theta_{t+1}\right) \tag{15}$$

8) Broadcast Global Model Output Phase: Over the 5G core network, the cloud server broadcasts the global model output $\hat{p}_{i,s}$ to every concentrator client.

9) FD Phase: The concentrator updates the local model based on the broadcasted global model output $\hat{p}_{i,s}$ and the local unlabeled data $D^{m,c}$. Specifically, the local model parameters are calculated as follows:

$$\theta_t = \theta_t - \eta\nabla\psi\left(D^{m,c}, \hat{p}_{i,s}|\theta_t\right) \tag{16}$$

The above communication process is iterated several times until the model converges, and the detailed process is shown in Algorithm 1.

C. DCGAN Intrusion Detection Model

Often, the data are unlabeled or unbalanced, and it is not practical to directly use the data in the model to make judgments. We use the DCGAN to construct intrusion detection models; this approach can solve the problem of data labeling, and can better extract dataset characteristics. The DCGAN is used for supervised learning in cloud servers and for local training, and updates the concentrators and cloud server concentrators. FD trains the DCGAN intrusion detection model collaboratively

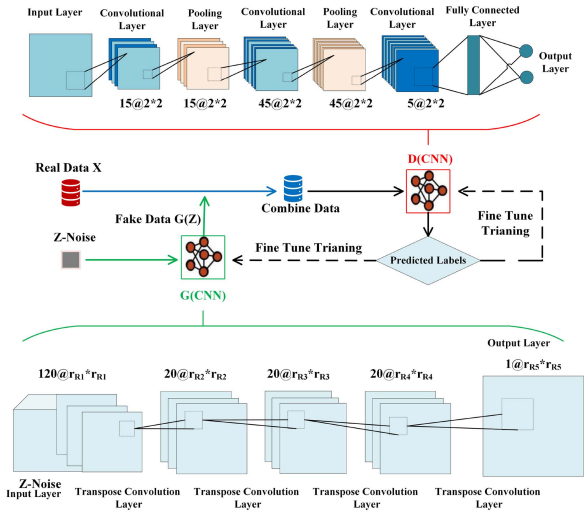

Fig. 4. Deep convolutional generative adversarial network.

Algorithm 2: DCGAN Algorithm.

Input: Input data $\mathrm{X_R}$; number of iterations L; label data $\mathrm{Y_{Y_z}}$,$\mathrm{Y_{Y_R}}$; comparison data $\mathrm{X_{X'_z}}$; generator G; discriminator D; threshold value υ,φ_1;

Output: Generator G'; discriminator D'.

Procedure:

Randomly generate $\mathrm{k_m}$ data between 0 and 1 to form the noise dataset X_z;

for $l = 1 : L$ **do**

- $\mathrm{X}'_z = G\left(\mathrm{X}_z\right)$;
- $X_c = [\mathrm{X}'_z, \mathrm{X}_R]$;
- $Y_c = D\left(X_c\right)$;
- The data $Y_c = \{Y_z, Y_R\}$ are processed as follows:
- $Y\left(Y_c\right) = \begin{cases} y_{1\iota=0}, y_{2\iota=1} if y_{1\iota} < y_{2\iota} \\ y_{1\iota=1}, y_{2\iota=0} if y_{1\iota} \geq y_{2\iota} \end{cases}, \iota = 1, ..., k_m + m_\chi$;
- $\varphi\left(Y\left(Y_z\right), Y_{Y_z}\right)$ and $\varphi\left(Y\left(Y_R\right), Y_{Y_R}\right)$ are processed as follows:
- $\varphi = \frac{TP+TN}{TP+TN+FP+FN}$;
- $simi\left(\mathrm{X'}_z, \mathrm{X}_{\mathrm{X'}_z}\right)$ are processed as follows:
- $simi\left(\mathrm{X'}_z, \mathrm{X}_{\mathrm{X'}_z}\right) = \frac{\sum_{\iota}^{k_m} simi\left(\mathrm{X'}_{z\iota}, \mathrm{X}_{\mathrm{X'}_{z\iota}}\right)}{k_m}$;
- **if** $\varphi\left(Y\left(Y_R\right), Y_{Y_R}\right) > \varphi_1$ **then**
  - $\varphi_1 = \varphi\left(Y\left(Y_c\right), [Y_{Y_z}, Y_{Y_R}]\right), D' = D$;
- **if** $simi\left(\mathrm{X'}_z, \mathrm{X}_{\mathrm{X'}_z}\right) > \nu$ **then**
  - $\nu = simi\left(\mathrm{X'}_z, \mathrm{X}_{\mathrm{X'}_z}\right), G' = G$;
- Let $\varphi_{Y_z} = \varphi\left(Y\left(Y_z\right), Y_{Y_z}\right)$;
- Let $\varphi_{Y_R} = \varphi\left(Y\left(Y_R\right), Y_{Y_R}\right)$;
- **if** $\varphi_{Y_z} < \varphi_{Y_R} \cap simi\left(\mathrm{X'}_z, \mathrm{X}_{\mathrm{X'}_z}\right) \leq \nu$ **then**
  - Train and update the generator G;
- **else**
  - Train and update the discriminator D;

return generator G'; discriminator D'.

over a 5G core network. A CNN discriminator and a CNN generator compose the DCGAN optimization model, which is gamed to produce the best outcomes. The goal of the discriminator is to identify anomalous data, whereas the generator's goal is to generate additional data to improve the classification. The DCGAN may also extract deep features from AMI network traffic via CNNs to increase the model's stability and training duration. The architecture of the DCGAN is shown in Fig. 4. The DCGAN is represented as follows:

$$\min_{G}\max_{D} V_{GAN}(G,D) = \mathrm{E}_{x\in \mathrm{X}}\left[Log\left(D(x)\right)\right] + \mathrm{E}_{\mathrm{z}\in \mathrm{X}_z}\left[\mathrm{Log}\left(1-D\left(G(x)\right)\right)\right] \quad (17)$$

In the suggested intrusion detection model based on the DCGAN, data z are selected from the dataset X_z with random noise as the input to the DCGAN generator. x_z and z are matrices of sizes $j_m \times k_m$ and $j_m \times k'_m$, respectively, where k_m and k'_m are the sample sizes of X_z and z, respectively, and where j_m is the feature size of the noisy data. The random noisy data are subsequently converted into fake data $X'_z = G(X_z)$ by the generator G, where X'_z is a matrix of size $k_m C \times C$. This process of generating false data from random, noisy data can be expressed as follows:

$$vX_z^l = g^l\left(\omega_G^l\right)^C \times X_z^{l-1} + b_G^l, l = 1,2,3,\ldots,n'_m \quad (18)$$

where X_z^0 denotes the input data z, $X_z^{n'_m}$ denotes the output X'_z of the generator G, X_z^l denotes the output of the lth layer, and $\theta_G^l = (\omega_G^l, b_G^l)$ denotes the parameters of the lth transposed convolutional layer. In addition, g^l is the activation function, which is typically set as the hyperbolic tangent function. The suggested strategy optimizes generator G via the mean square error (MSE) function to obtain the best result for the generator network. The MSE loss function can be expressed as follows:

$$\xi_{MSE}\left(Q_L, Q'_L\right) = \frac{\sum_{l=1}^{e}\left(Q_{Ll} - Q'_{Ll}\right)^2}{e} \quad (19)$$

To train the discriminator network, the generator-generated dataset X'_z is combined with the input dataset $X_R = (X_1, X_2, \ldots, X_{m_x})$ to generate the combined data X_c. where m_x is the number of samples of genuine data, $X_i (i = 1,2,3,\ldots,m_x)$ is the data matrix, and X_c is a matrix of size $k_m + m_x$. The discriminator network is trained using the combined data X_c. Discriminator D is a CNN network used to generate discriminative data $Y_c = D(X_c)$, and Y_c is a matrix of size $2 \times (k_m + m_x)$. The training process of discriminator D is shown in Formula (22).

$$\psi_*^l\left(k_{cnn}^l * X^{(l-1)} + b_{cnn}^l\right) = X^l, l = 1,2,\ldots,n_C \quad (20)$$

where X^0 denotes the input data X, X^{n_C} denotes the output data Y, and where X^l denotes the output of the lth layer. The maximum pooling operation is used by all pooling levels. Additionally, $\theta_{cnn}^l = (k_{cnn}^l, b_{cnn}^l)$ is used to denote the parameters of the lth convolutional layer. The activation function ψ_*^l can be a hyperbolic tangent function, a ReLU function, or a Softmax function. The cross-entropy loss function ϕ is used as the loss function, and is defined as follows:

$$\phi\left(Q_L, Q'_L\right) = \frac{1}{e}\sum_{l}^{e} -\left[Q'_{Ll} log\left(Q_{Ll}\right) + \left(1 - Q'_{Ll}\right) log\left(1 - Q_{Ll}\right)\right] \quad (21)$$

where the lth actual value in a batch of input data is indicated by the symbol Q'_{Ll}; in a batch of input data, e represents the number of samples, and Q_{Ll} represents the lth neural network predictor. The cross-entropy loss function can be used to improve the CNN performance effectively.

Additionally, the loss function of the model can be improved via the adaptive moment estimation (ADAM) optimization function with the following formula:

$$g_t = \frac{1}{e}\nabla\theta\sum_{k=1}^{e}\phi\left(Q_L, Q'_L\right) \quad (22)$$

$$m_t = \rho_1 m_{t-1} + (1-\rho_1) g_t \quad (23)$$

$$n_t = \rho_2 n_{t-1} + (1-\rho_2) g_t^2 \quad (24)$$

$$\hat{m}_t = \frac{m_t}{1-\rho_1^t} \quad (25)$$

$$\hat{n}_t = \frac{n_t}{1-\rho_2^t} \quad (26)$$

$$\Delta\theta = -\eta\frac{\hat{m}_t}{\sqrt{\hat{n}_t} + \varepsilon} \quad (27)$$

where ϕ denotes the loss function of the CNN, and where $\nabla\theta$ denotes the gradient of the parameter θ. g_t denotes the gradient of the tth time step, and t denotes the number of optimization time steps. ρ_1 and ρ_2 denote the exponential decay rates of the moment estimate, and the values of ρ_1 and ρ_2 lie between $[0,1]$, respectively, indicating the decay degree of the moment estimate. In addition, m_t and n_t denote the biased first-order moment estimates and second-order moment estimates, respectively. Correspondingly, $\hat{m}_t$ and $\hat{n}_t$ denote the bias-corrected first-order moments and bias-corrected second-order moments, respectively. ε denotes the constant used for numerical stabilization. $\Delta\theta$ denotes the magnitude of the updated parameter θ.

$$Y\left(Y_c\right) = \begin{cases} y_{1\iota}=0, y_{2\iota}=1 if y_{1\iota} < y_{2\iota} \\ y_{1\iota}=1, y_{2\iota}=0 if y_{1\iota} \geq y_{2\iota} \end{cases}, [4pt]\iota = 1,\ldots,k_m + m_\chi \quad (28)$$

where y_{1l} and y_{2l} denote the discriminant results of the lth data point of the discriminator. Y_z and Y_R compose the discriminator's output data Y_c. The discriminant result of the produced data X'_z is Y_z, and the discriminant result of the original data X_R is Y_R. The function φ is subsequently used to determine Y_z and Y_R. The function φ can be used to determine the discriminative ability of the original data and the created data, and the following technique is used to implement the function φ:

$$\varphi = \frac{TP + TN}{TP + TN + FP + FN} \quad (29)$$

where the letters TP, FP, FN, and TN represent true-positive, false-positive, false-negative, and true-positive samples, respectively. The matrices $corr(A, B)$ and their similarity are defined as follows:

$$corr(A,B) = \frac{\sum_{m_s}\sum_{n_s}\left(a_{m_s,n_s}-\bar{a}\right)\left(b_{m_s,n_s}-\bar{b}\right)}{\sqrt{\left(\sum_{m_s}\sum_{n_s}\left(a_{m_s,n_s}-\bar{a}\right)^2\right)\left(\sum_{m_s}\sum_{n_s}\left(b_{m_s,n_s}-\bar{b}\right)^2\right)}} \tag{30}$$

where $\bar{a}$ and $\bar{b}$ represent the average values of A and B, respectively, and both matrices A and B are of size $m_s \times n_s$. Consequently, the following formula can be used to determine how similar X'_z and $X_{X'_z}$ are on average:

$$Cr\left(X'_z, X_{X'_z}\right) = \frac{\sum_l^{k_c} corr\left(X'_{zl}, X_{X'_{zl}}\right)}{k_c} \tag{31}$$

where the generated data X'_z are the generated data, and where $X_{X'_z}$ is the comparison data. The model is updated by determining the sizes of $\varphi(Y(Y_z), Y_{Y_z})$ and $\varphi(Y(Y_R), Y_{Y_R})$ and determining whether the The $Cr(X'_z, X_{X'_z})$ value reaches the threshold υ. The labels for the produced data X'_z and the original data X_R are Y_{Y_z} and Y_{Y_R}, respectively. If $\varphi(Y(Y_z), Y_{Y_z})$ is smaller than $\varphi(Y(Y_R), Y_{Y_R})$ and $Cr(X'_z, X_{X'_z}) \leq \nu$, then the generator is updated; otherwise, the discriminator is updated. Implementing the Nash equilibrium yields the ideal discriminator. The precise method is expressed as follows:

$$\max DE_{X\in X_R}\left[log\left(\varphi\left(D(X)\right)\right)\right] + E_{z\in X_z} \left[log\left(1-\varphi\left(D\left(G(z)\right)\right)\right)\right] \tag{32}$$

Finding the best discriminator is the objective function. The value function is therefore expressed as follows:

$$V_{GAN}(G,D) = E_{X\in X_R}\left[log\left(\varphi\left(D(X)\right)\right)\right] + E_{z\in X_z}\left[log\left(1-\varphi\left(D\left(G(z)\right)\right)\right)\right] \tag{33}$$

If G is constant, then $D^*_G = arg\max DV_{GAN}(G, D)$ can be considered the best discriminator. Accordingly, if D is a constant, the optimal generator $G^*_G = arg\min GV_{GAN}(G, D^*_G)$ can be obtained. After a series of training steps, the optimal discriminator $D^*_G = arg\max DV_{GAN}(G^*_G, D)$ can be obtained. The details of the $DCGAN$ algorithm are shown in Algorithm 2.

V. Experimental Evaluation and Analysis

A. Experimental Setup

1) Environment Setting: We formulate the federated semi-supervised learning intrusion detection model via the PyTorch framework in Python 3.8 version 1.9.0. The experiments were executed on a Linux operating system with an Intel Core i5-6200U CPU and an AMD Radeon (TM) R5 M430 GPU. In the experiments, 100 concentrator clients are considered, and the training set is divided equally and randomly into 100 copies, with one copy assigned to each client. The rounds are set as the number of test rounds and kept constant for each set of experiments for accurate comparisons. For each global round, the local model output of the concentrator is transmitted to the cloud server via the 5G core network for aggregation and updating. We perform different parameter selection and training steps, thus presetting the learning rate, batch size, and local training epochs for each communication round to 0.0001, 50, and 10, respectively.

2) NSL-KDD Dataset: This dataset has been widely used in AMI intrusion detection [8], [9], [10]. It contains all types of attacks that AMI networks can be subjected to, including the 4 attack categories DoS, Probe, U2R, and R2L, and the 39 attack subcategories, making it an ideal dataset for evaluating the performance of IDS models.

The data distribution of the NSL-KDD dataset has a periodic character, which is critical for detecting and defending against cyberattacks. Each piece of data contains 42 features, of which 38 are numeric, 3 are symbolic, and 1 is a label [27]. The training and testing intrusion detection models can use these features to help detect and respond to various cyberattacks promptly. Table II details the 4 attack categories.

TABLE II
Numerical Processing of Label Data

Parameter	Configuration
Normal	(1, 0, 0, 0, 0)
U2R	(0, 1, 0, 0, 0)
R2L	(0, 0, 1, 0, 0)
Probe	(0, 0, 0, 1, 0)
DoS	(0, 0, 0, 0, 1)

3) Data Preprocessing: The NSL-KDD dataset includes samples of multiple data types, including symbolic data, and neural networks cannot directly process such samples. Therefore, to better train the model, preprocessing operations such as numerical processing, one-hot coding, normalization, and balancing sample categories must first be performed on the data to improve the accuracy.

(1) Numerical Processing and One-Hot Coding: The goal is to convert symbolic features to numerical features for numerical processing and institute one-hot encoding during data preprocessing. The main features that need to be processed by numerical and one-hot encoding are protocol_type, service, and flag. The protocol type consists of three attributes, namely, TCP, UDP, and ICMP, which are represented by 1x3-dimensional vectors (0,0,1), (0,1,0), and (1,0,0), respectively, after numerical encoding and one-hot encoding. The service and sign features contain 70 and 11 attributes, respectively, and can be represented by 1x70 and 1x11 dimensional vectors, respectively. After numerical and one-hot encoding of these three symbolic features, we obtain a 1x84-dimensional numerical feature vector. This vector is combined with the original 1x38-dimensional numerical feature vector to obtain a 1x122-dimensional numerical feature vector. In addition, the symbolic label data must be numerically processed. This dataset has five label types, namely, labels representing normal behavior (Normal) and attack behavior (U2R, R2L, Probe, DoS). Table IV presents the numerical processing results of the labels.

(2) Normalization Processing: After the numerical processing and one-hot coding process, certain numerical features may

TABLE III
COMPARISON OF THE DETECTION PERFORMANCE WITH THE CENTRALIZED METHODS

Methods	Precision(%)	Recall(%)	F1(%)
CNN [30]	94.52	94.85	94.68
GA-DBN [31]	97.36	97.67	97.51
GAN [32]	93.24	94.73	93.98
BiLSTM [33]	99.05	90.79	94.74
FSSL-IDS	99.54	99.50	99.52

TABLE IV
ACCURACY, FPR, AND PRIVACY PROTECTION COMPARED TO THE EXISTING METHODS

Methods	Dataset(%)	Accuracy(%)	FPR(%)	Privacy
NBTree [34]	NSL-KDD	94.60	5.40	✗
DBN-ELM [35]	NSL-KDD	97.82	1.81	✗
A-DQN [36]	NSL-KDD	97.20	1.24	✗
5-layer AE [37]	NSL-KDD	90.61	- -	✗
GA-ELM [16]	KDD-CUP99	98.90	1.36	✗
AE-CNN [17]	KDD-CUP99	93.99	7.00	✗
HFL [25]	NSL-KDD	99.48	- -	✓
FLDNN [26]	NSL-KDD	99.55	1.21	✓
Fed_ADBN [10]	NSL-KDD	99.14	1.09	✓
FDNN [27]	NSL-KDD	99.32	- -	✓
FSSL-IDS	NSL-KDD	99.58	1.02	✓

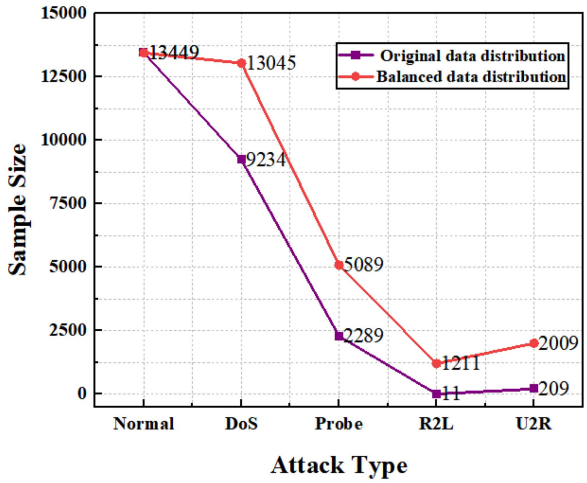

Fig. 5. Balancing treatment of small-sample attack classes in the NSL-KDD training set.

mislead the learning model if used directly due to their large values. The purpose of normalization is to improve the learning models' performance. Normalization refers to mapping the datasets' numerical features into a uniform interval [0,1] range. The specific normalization formula is as follows:

$$x = \frac{x_i - x_{\min}}{x_{\max} - x_{\min}} \tag{34}$$

where x_i denotes the feature's starting value; $x_{\max}$ and $x_{\min}$ denote the feature's highest and lowest values, respectively; and x denotes the normalization outcome.

(3) Sparse Class Balancing: Fig. 5 shows the sample sizes for different attack types. Owing to the uneven distribution of attack categories and unbalanced labels in the NSL-KDD dataset, it is necessary to balance the data. Through preprocessing, the sample data generated are the same as the original sample, and the final data sample exhibits a balanced state. The sample size of the normal attack type is the greatest, and that of the R2L attack type is the lowest.

4) Evaluation Metrics: In this work, the accuracy, precision, recall, F1, false-positive rate (FPR), and loss were used as evaluation metrics. These metrics are calculated from the true positive (TP), false-negative (FN), false-positive (FP), and true negative (TN) values, where TP denotes the number of positive class samples correctly predicted as positive, FN denotes the number of positive class samples incorrectly predicted as attack samples, FP denotes the number of attack samples incorrectly predicted as positive, and TN denotes the number of attack samples correctly predicted as attack samples.

B. Performance Comparison with the Centralized Detection Scheme

We compare the effectiveness of the proposed FSSL-IDS approach with that of the centralized intrusion detection strategy. In this set of experiments, 100 concentrator clients are assumed to be distributed in the AMI system, and each concentrator represents an IDS that monitors the network traffic of the AMI network. According to the experimental results in Fig. 6(a), the loss of the FSSL-IDS approach is 0.06 under 100 communication rounds, which is significantly lower than that of the centralized scheme, with a loss value reduction of approximately 50%. In addition, according to the experimental results in Fig. 6(b), the FSSL-IDS method achieves a 99.58% detection accuracy over 100 communication rounds, which is an improvement of approximately 2% over that of the traditional centralized method; this is because our method uses a DCGAN model, which makes it easier for neural networks to converge during training by processing datasets with complete and balanced dataset labels.

Specifically, the FSSL-IDS method achieves a better performance in monitoring AMI network traffic. With distributed monitoring, each concentrator client can effectively detect intrusions, thus reducing the risk of system attacks. The significant loss reduction in the FSSL-IDS method indicates that the method can better control the complexity of the model without sacrificing detection accuracy. Additionally, the detection accuracy of the FSSL-IDS method is 2% greater than that of the traditional centralized method, which indicates that the method can detect intrusions more accurately, and thus, better secure the AMI systems.

Table III compares the precision, recall, and F1 score of the FSSL-IDS method with those of the centralized method. Table III shows that the proposed FSSL-IDS outperforms the centralized method on all three evaluation metrics. Specifically, the FSSL-IDS method achieves 99.54%, 99.50%, and 99.52% values under the precision, recall, and F1 metrics, respectively, which represents improvements of 0.49%-6.32% in precision, 1.83%-8.71% in recall, and 2.01%-5.54% in F1 over the centralized method. This indicates that the FSSL-IDS approach performs better at distinguishing natural traffic from attack traffic, thus achieving more accurate detection of intrusions and protecting the AMI systems. In addition, the superiority of the FSSL-IDS method is shown in Fig. 7. The figure shows that the

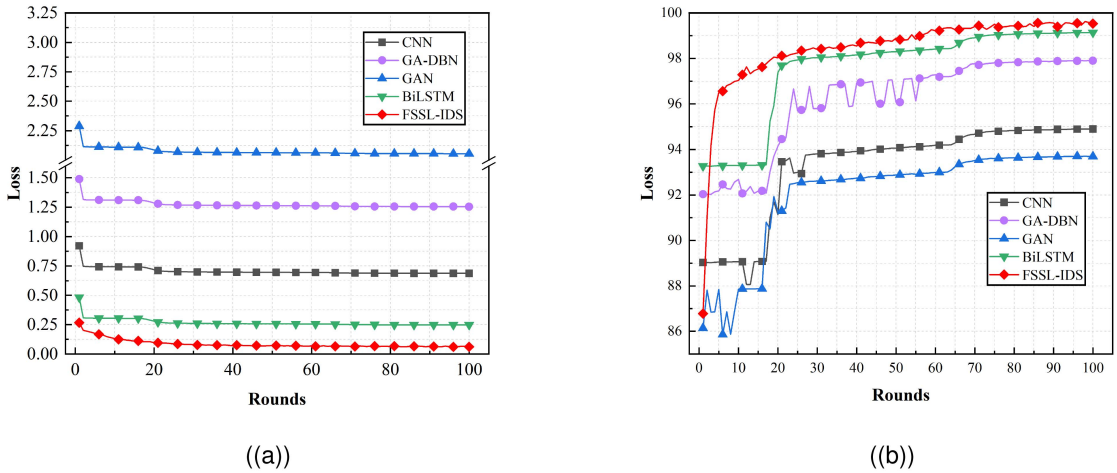

Fig. 6. Comparison with centralized detection methods (a) loss (b) accuracy.

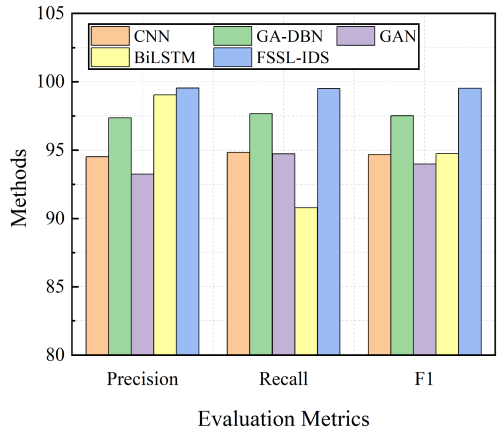

Fig. 7. Precision, recall, and F1 comparisons with centralized detection methods.

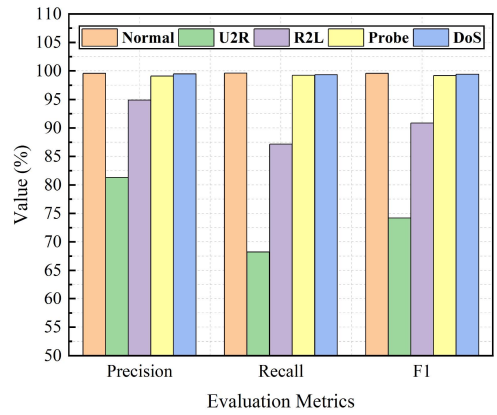

Fig. 8. Multiclassification results of the FSSL-IDS method.

FSSL-IDS approach can better distinguish between normal and attack traffic conditions, resulting in higher detection accuracy and a lower false-positive rate.

In addition, we analyze the detection performance of the proposed method for each attack category. As shown in Fig. 8, the FSSL-IDS approach achieves better performances for normal, DoS, and probe attacks, and higher detection accuracies for R2L and U2R attacks; this is because the proposed FSSL-IDS model uses an innovative method of data feature extraction. Excellent dataset preprocessing and multilayer convolution make the extracted features more representative and more convenient for neural network fitting. Therefore, we can draw two conclusions: (1) the FSSL-IDS approach has been studied and is verified to achieve better detection performance and privacy protection than the centralized approach, and (2) we can achieve multicategory detection performance, which meets the high detection performance requirements for the AMI networks.

C. Performance Comparison With Existing Detection Schemes

To evaluate the effectiveness of the proposed method's detection, we compared the proposed method's performance with that of the existing detection methods, namely, the NBTree [32], DBN-ELM [33], A-DQN [34], 5-layer AE-based [35], AE-CNN-based [17], HFL-Transformer [8], FLDNN [9], and Fed_ADBN [10] methods, via the same evaluation metrics.

According to the comparison results in Table IV, the proposed FSSL-IDS method yields better accuracy and a lower false alarm rate than the other methods. The accuracy of the FSSL-IDS method is 99.58%, which is 0.1%-9% better than that of the existing methods. The false alarm rate of the FSSL-IDS method is 1.02, which is 0.07%-6% lower than that of the existing methods. The lower the FPR is, the better the classification performance. In addition, the FSSL-IDS method has a significant of privacy protection advantage over the existing centralized intrusion detection methods; this is because only our method is trained using unlabeled data, and a comprehensive dataset is more likely to reveal the most realistic AMI attack patterns. In the table, "–" indicates that no results are available.

D. Performance Comparison With AMI Federated Detection Methods

In addition to the above experiments, we perform a comprehensive comparison using different evaluation metrics. Table V shows the performance comparison of the proposed FSSL-IDS

TABLE V
PERFORMANCE COMPARISON OF THE EXISTING AMI NETWORK FEDERATED DETECTION METHODS

Methods	Precision(%)	Recall(%)	F1(%)
HFL-Transformer [25]	99.49	99.49	99.49
FLDNN [26]	99.50	99.46	99.48
Fed_ADBN [10]	99.24	99.31	99.28
FSSL-IDS	99.54	99.50	99.52

TABLE VI
MULTI-CLASSIFICATION PRECISION COMPARISON OF THE EXISTING AMI NETWORK FEDERATED DETECTION METHODS

	Methods	HFL	FLDNN	FDNN	FSSL-IDS
Precision	DoS	99.85	100.0	99.96	99.48
	Probe	98.76	99.00	98.84	99.11
	R2L	87.21	89.00	93.73	94.88
	U2R	78.57	75.00	80.00	81.32

TABLE VII
MULTI-CLASSIFICATION RECALL COMPARISON OF THE EXISTING AMI NETWORK FEDERATED DETECTION METHODS

	Methods	HFL	FLDNN	FDNN	FSSL-IDS
Recall	DoS	99.91	100.0	99.38	99.35
	Probe	98.85	98.00	99.38	99.24
	R2L	87.55	85.00	86.33	87.15
	U2R	44.00	50.00	67.23	68.23

method with other methods in terms of precision, recall, and F1 score. According to the data in Table V, the proposed FSSL-IDS method performs well on all the evaluation metrics, with a detection precision of 99.5%, a recall rate of 99.50%, and an F1 score of 99.52%. These data indicate that the proposed method outperforms the existing AMI network-federated detection methods in terms of detection performance.

Additionally, Tables VI and VII present the classification performances of the proposed FSSL-IDS approach and current AMI network federated detection methods on several threat categories. These data further show that the classification performance of the proposed method is significantly better than that of the existing AMI network federated detection methods.

According to the experimental results in Tables VI and VII, the proposed FSSL-IDS method has almost the same accuracy and recall rate as other models in the face of DoS attacks and probe attacks. It significantly improves the detection performance of R2L attacks and U2R attacks.

The proposed method exhibits excellent performance in identifying various network attacks, particularly R2L and U2R attacks, by fully leveraging unlabeled data from the local concentrator. This significantly enhances the precision of detecting small-sample attacks; this is because, compared with other models, our model processes the dataset in a way that makes it difficult for noise to affect the detected data.

E. Communications Overhead Evaluation

In this section, we compare the performance of the federal learning approach and the proposed approach for different model sizes in respect to the communication overhead per round. Table VIII shows that the proposed FSSL-IDS approach outperforms FL in terms of communication overhead, regardless of model size. As the model size increases, the FL overhead increases due to the need to upload model parameters, and the parameters are proportional to the model size. In contrast, the FSSL-IDS uploads model outputs that do not depend on the model size, thereby reducing communication overhead and enhancing data security. This method avoids potential leakage of sensitive information by not requiring uploading local model parameters, ensuring efficient distributed learning with strong privacy protections.

TABLE VIII
COMPARISON OF COMMUNICATION OVERHEAD PER ROUND FOR DIFFERENT MODEL SIZES OF FEDERATED LEARNING METHODS

Methods	Communication overhead	Cost ratio
FL(MLP)	156MB	208%
FL(CNN)	768MB	1024%
FSSL-IDS(MLP)	75MB	100%
FSSL-IDS(CNN)	75MB	100%
FSSL-IDS(DCGAN)	75MB	100%

VI. CONCLUSION

This paper investigates efficient intrusion detection based on federated semi-supervised learning in AMI systems. First, we design an FD-based intrusion detection framework to reduce the communication overhead. Next, we use an effective intrusion detection algorithm to improve the classification performance. Finally, we design a DCGAN model to improve the sample data quality. The use of a federated semi-supervised algorithm allows for improvements of more than 99.58% and 6% in accuracy and the false-positive rate, respectively.

REFERENCES

[1] E. Natalizio, H. Ishii, R. Lu, and S. Pack, "Introduction to the special section on security and privacy of smart network systems," *IEEE Trans. Netw. Sci. Eng.*, vol. 8, no. 3, pp. 1975–1977, Jul.–Sep. 2021.
[2] Y. Wang, Q. Chen, T. Hong, and C. Kang, "Review of smart meter data analytics: Applications, methodologies, and challenges," *IEEE Trans. Smart Grid*, vol. 10, no. 3, pp. 3125–3148, May 2019.
[3] B. Goswami, R. Jurdak, and G. Nourbakhsh, "Node allocation strategy for low latency neighborhood area networks in smart grid," *IEEE Trans. Netw. Sci. Eng.*, vol. 11, no. 5, pp. 5087–5098, Sep./Oct. 2024.
[4] J. Bi, F. Luo, G. Liang, X. Yang, S. He, and Z. Y. Dong, "Impact assessment and defense for smart grids with FDIA against AMI," *IEEE Trans. Netw. Sci. Eng.*, vol. 10, no. 2, pp. 578–591, Mar./Apr. 2023.
[5] D. E. Whitehead, K. Owens, D. Gammel, and J. Smith, "Ukraine cyber-induced power outage: Analysis and practical mitigation strategies," in *Proc. 70th Annu. Conf. Protective Relay Engineers*, Apr. 2017, pp. 1–8.
[6] V. Kayalvizhy and A. Banumathi, "A survey on cyber security attacks and countermeasures in smart grid metering network," in *Proc. 5th Int. Conf. Comput. Methodol. Commun.*, Apr. 2021, pp. 160–165.
[7] G. Lu and X. Tian, "An efficient communication intrusion detection scheme in AMI combining feature dimensionality reduction and improved LSTM," *Secur. Commun. Netw.*, vol. 2021, no. 1, 2021, Art. no. 6631075.
[8] X. Sun et al., "A hierarchical federated learning-based intrusion detection system for 5G smart grids," *Electronics*, vol. 11, no. 16, 2022, Art. no. 2627.
[9] P. H. Mirzaee, M. Shojafar, Z. Pooranian, P. Asefy, H. Cruickshank, and R. Tafazolli, "Fids: A federated intrusion detection system for 5G smart metering network," in *Proc. 17th Int. Conf. Mobility, Sens. Netw.*, Dec. 2021, pp. 215–222.
[10] Z. Xia et al., "Fed_ADBN: An efficient intrusion detection framework based on client selection in AMI network," *Expert Syst.*, vol. 40, no. 4, 2023, Art. no. e12983.
[11] F. Sattler, A. Marban, R. Rischke, and W. Samek, "CFD: Communication-efficient federated distillation via soft-label quantization and delta coding," *IEEE Trans. Netw. Sci. Eng.*, vol. 9, no. 4, pp. 2025–2038, Jul./Aug. 2022.

[12] S. Itahara, T. Nishio, Y. Koda, M. Morikura, and K. Yamamoto, "Distillation-based semi-supervised federated learning for communication-efficient collaborative training with non-IID private data," *IEEE Trans. Mobile Comput.*, vol. 22, no. 1, pp. 191–205, Jan. 2023.
[13] D. Sui, Y. Chen, J. Zhao, Y. Jia, Y. Xie, and W. Sun, "Feded: Federated learning via ensemble distillation for medical relation extraction," in *Proc. Conf. Empirical Methods Natural Lang. Process.*, Nov. 2020, pp. 2118–2128.
[14] S. Bhattacharjee, A. Thakur, and S. K. Das, "Towards fast and semi-supervised identification of smart meters launching data falsification attacks," in *Proc. Asia Conf. Comput. Commun. Secur.*, May 2018, pp. 173–185.
[15] R. Sharma, A. M. Joshi, C. Sahu, G. Sharma, K. T. Akindeji, and S. Sharma, "Semi supervised cyber attack detection system for smart grid," in *Proc. 30th Southern Afr. Universities Power Eng. Conf.*, Jan. 2022, pp. 1–5.
[16] K. Zhang, Z. Hu, Y. Zhan, X. Wang, and K. Guo, "A smart grid AMI intrusion detection strategy based on extreme learning machine," *Energies*, vol. 13, no. 18, 2020, Art. no. 4907.
[17] Z. Zheng, Y. Yang, X. Niu, H. -N. Dai, and Y. Zhou, "Wide and deep convolutional neural networks for electricity-theft detection to secure smart grids," *IEEE Trans. Ind. Informat.*, vol. 14, no. 4, pp. 1606–1615, Apr. 2018.
[18] S. M. Taghavinejad, M. Taghavinejad, L. Shahmiri, M. Zavvar, and M. H. Zavvar, "Intrusion detection in IoT-based smart grid using hybrid decision tree," in *Proc. 6th Int. Conf. Web Res.*, Apr. 2020, pp. 152–156.
[19] R. Vijayanand, D. Devaraj, and B. Kannapiran, "A novel deep learning based intrusion detection system for smart meter communication network," in *Proc. IEEE Int. Conf. Intell. Techn. Control, Optim. Signal Process.*, Apr. 2019, pp. 1–3.
[20] Y. He, G. J. Mendis, and J. Wei, "Real-time detection of false data injection attacks in smart grid: A deep learning-based intelligent mechanism," *IEEE Trans. Smart Grid*, vol. 8, no. 5, pp. 2505–2516, Sep. 2017.
[21] Y. Xiao, C. Xing, T. Zhang, and Z. Zhao, "An intrusion detection model based on feature reduction and convolutional neural networks," *IEEE Access*, vol. 7, pp. 42210–42219, 2019.
[22] C. Xu, J. Shen, X. Du, and F. Zhang, "An intrusion detection system using a deep neural network with gated recurrent units," *IEEE Access*, vol. 6, pp. 48697–48707, 2018.
[23] B. McMahan, E. Moore, D. Ramage, S. Hampson, and B. A. Y. Arcas, "Communication-efficient learning of deep networks from decentralized data," in *Proc. Int. Conf. Artif. Intell. Statist.*, Apr. 2017, pp. 1273–1282.
[24] S. A. Rahman, H. Tout, C. Talhi, and A. Mourad, "Internet of things intrusion detection: Centralized, on-device, or federated learning?," *IEEE Netw.*, vol. 34, no. 6, pp. 310–317, Nov./Dec. 2020.
[25] H. Liang, D. Liu, X. Zeng, and C. Ye, "An intrusion detection method for advanced metering infrastructure based on federated learning," *J. Modern Power Syst. Clean Energy*, vol. 11, no. 3, pp. 927–937, May 2023.
[26] M. Wen, R. Xie, K. Lu, L. Wang, and K. Zhang, "FedDetect: A novel privacy-preserving federated learning framework for energy theft detection in smart grid," *IEEE Internet Things J.*, vol. 9, no. 8, pp. 6069–6080, Apr. 2022.
[27] S. Revathi and A. Malathi, "A detailed analysis on NSL-KDD dataset using various machine learning techniques for intrusion detection," *Int. J. Eng. Res. Technol.*, vol. 2, no. 12, pp. 1848–1853, 2013.
[28] R. Vinayakumar, K. P. Soman, and P. Poornachandran, "Applying convolutional neural network for network intrusion detection," in *Proc. Int. Conf. Adv. Comput., Commun. Inform.*, Sep. 2017, pp. 1222–1228.
[29] N. Gao, L. Gao, Q. Gao, and H. Wang, "An intrusion detection model based on deep belief networks," in *Proc. 2nd Int. Conf. Adv. Cloud Big Data*, Nov. 2014, pp. 247–252.
[30] G. Xie, L. T. Yang, Y. Yang, H. Luo, R. Li, and M. Alazab, "Threat analysis for automotive CAN networks: A GAN model-based intrusion detection technique," *IEEE Trans. Intell. Transp. Syst.*, vol. 22, no. 7, pp. 4467–4477, Jul. 2021.
[31] J. Sinha and M. Manollas, "Efficient deep CNN-BiLSTM model for network intrusion detection," in *Proc. 3rd Int. Conf. Artif. Intell. Pattern Recognit.*, Jun. 2020, pp. 223–231.
[32] D. H. Deshmukh, T. Ghorpade, and P. Padiya, "Improving classification using preprocessing and machine learning algorithms on NSL-KDD dataset," in *Proc. Int. Conf. Commun., Inf., Comput. Technol.*, Jan. 2015, pp. 1–6.
[33] D. Liang and P. Pan, "Research on intrusion detection based on improved DBN-ELM," in *Proc. Int. Conf. Commun., Inf. Syst., Comput. Eng.*, Jul. 2019, pp. 495–499.
[34] Z. Liu, L. Hou, K. Zheng, Q. Zhou, and S. Mao, "A DQN-based consensus mechanism for blockchain in IoT networks," *IEEE Internet Things J.*, vol. 9, no. 14, pp. 11962–11973, Jul. 2022.
[35] R. Yao, N. Wang, Z. Liu, P. Chen, D. Ma, and X. Sheng, "Intrusion detection system in the smart distribution network: A feature engineering based AE-LightGBM approach," *Energy Reports*, vol. 7, pp. 353–361, 2021.
[36] H. Alshede, L. Nassef, N. Alowidi, and E. Fadel, "Ensemble voting-based anomaly detection for a smart grid communication infrastructure," *Intell. Automat. Soft Comput.*, vol. 36, no. 3, pp. 3257–3278, 2023.
[37] A. Amjad, M. A. Mohammed, M. H. Ahmed, I. T. Ahmed, M. A. E. Rasha, and H. S. Ahmed, "Implementation of african vulture optimization algorithm based on deep learning for cybersecurity intrusion detection," *Alexandria Eng. J.*, vol. 79, no. 105–115, pp. 1110–0168, 2023.
[38] A. Al-Abassi, A. N. Jahromi, H. Karimipour, A. Dehghantanha, P. Siano, and H. Leung, "A self-tuning cyber-attacks' location identification approach for critical infrastructures," *IEEE Trans. Ind. Informat.*, vol. 18, no. 7, pp. 5018–5027, Jul. 2022.
[39] J. Wang, Z. Q. Xia, Y. L. Chen, C. Hu, and F. Yu, "Intrusion detection framework based on homomorphic encryption in AMI network," *Front. Phys.*, vol. 10, 2022, Art. no. 1102892.
[40] F. Naeem, M. Ali, and G. Kaddoum, "Federated-learning-empowered semi-supervised active learning framework for intrusion detection in ZSM," *IEEE Commun. Mag.*, vol. 61, no. 2, pp. 88–94, Feb. 2023.

Zhuoqun Xia received the Ph.D. degree from the School of Information Science and Engineering, Central South University, Changsha, China, in 2012. In 2000, he joined the School of Computer and Communication Engineering, Changsha University of Science and Technology, Changsha, where he is currently a Full Professor. He has authored or coauthored more than 50 research papers in journals or conferences. His research interests include cryptography, network and information security.

Haidong Tang received the B.Sc. degree in network engineering from Dalian Minzu University, Dalian, China, in 2022. He is currently working toward the master's degree in computer science from the Changsha University of Science and Technology, Changsha, China. His research interests include cryptography, network and information security, especially the security of the smart grid.

Zhenzhen Hu (Member, IEEE) received the Ph.D. degree in computer science from Hunan University, Changsha, China, in 2022. She is currently a Teacher of computer science with the University of South China, Hengyang, China. Her research interests include mobile and wireless networks, especially UAV wireless communication in wireless network.

Hongmei Zhou (Member, IEEE) received the master's degree in computer science from the Changsha University of Science and Technology, Changsha, China, in 2023. Her research interests include cryptography, network and information security, especially the security of the AMI system.